

MINI-PROJECT REPORT

Adaptive WebRTC Video Conferencing with NAT Traversal

A Study, Design and Implementation of Signaling, ICE/NAT Traversal
(STUN/TURN),
and Network-Adaptive Media Transmission for Real-Time Communication

Submitted by

Simiyon Vinscent Samuel – 3122247001062

Sundareswaran Rajaguru – 3122247001048

Vijayaraaghavan K S – 3122247001066

Department of Computer Science and Engineering

SSN College of Engineering

Academic Year 2026–2027

Contents

Abstract	1
1 Introduction	2
1.1 Background	2
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope of the Project	3
1.5 Motivation	3
2 Literature / Background Study	4
2.1 Transport for Real-Time Media: Why Not Plain TCP	4
2.2 STUN (RFC 5389)	4
2.3 TURN (RFC 8656 / legacy RFC 5766)	4
2.4 ICE (RFC 8445)	4
2.5 SDP Offer/Answer and Signaling	5
2.6 Congestion / Quality Adaptation	5
2.7 Transport Security: HTTPS and the Secure-Context Requirement	5
3 System Analysis	6
3.1 Existing System	6
3.2 Limitations of Existing (Minimal) Systems	6
3.3 Proposed System	6
3.4 Advantages of Proposed System	7
4 System Design	8
4.1 System Architecture	8
4.2 Components / Modules	8
4.3 Data Flow	9
5 Implementation	10
5.1 Technologies Used	10
5.2 Hardware Requirements	10
5.3 Software Requirements	10
5.4 Module Implementation	10
5.4.1 Signaling Server	10
5.4.2 TURN Credential Proxy	11
5.4.3 Secure Transport Bootstrap	11
5.4.4 ICE / NAT-Traversal Configuration	12
5.4.5 Adaptive Quality Controller	12

5.5	Algorithms / Protocols	13
5.5.1	ICE Candidate Gathering and Pairing (RFC 8445)	13
5.5.2	Adaptive Quality State Machine	13
6	Experimental Setup	14
6.1	Experiment 1: Direct Peer-to-Peer Path	14
6.2	Experiment 2: TURN-Relay Path Characterization	14
7	Results and Discussion	16
7.1	Experimental Results	16
7.1.1	Experiment 1 – Direct P2P (measured via live <code>getStats()</code>)	16
7.1.2	Experiment 2 – TURN Relay Path (measured network RTT to the relay)	16
7.2	Graphical Analysis	16
7.3	Discussion	16
8	Conclusion	19
9	Future Enhancement	20
	References	21

Abstract

Real-time peer-to-peer video conferencing over the public Internet is complicated by two fundamental networking problems: (1) most end hosts sit behind NAT devices and firewalls that block unsolicited inbound connections, and (2) the available path bandwidth, round-trip time (RTT), and packet loss between two hosts vary continuously and unpredictably, so a fixed-quality media stream either wastes bandwidth or stalls the call. Browsers additionally require a *secure context* (HTTPS or `localhost`) before they grant an application access to the camera and microphone at all, which is easy to miss when testing across a local network.

This project addresses all three problems in a single, working, and reproducible system: a mesh WebRTC video-conferencing web application. A lightweight Node.js/Express server distributes signaling messages (SDP offers/answers, ICE candidates, and room membership) over WebSocket, while media itself flows directly between browsers using the WebRTC `RTCPeerConnection` API. NAT traversal is handled through the Interactive Connectivity Establishment (ICE) framework, combining public STUN servers with a credentialed TURN relay (Metered.ca) that the server exposes to clients through a proxy endpoint so the relay API key is never shipped to the browser. A client-side adaptive controller polls `RTCPeerConnection.getStats()` every three seconds, computes RTT, packet loss, and bitrate, and drives a four-tier quality state machine (`HIGH` $\rightarrow$ `MEDIUM` $\rightarrow$ `LOW` $\rightarrow$ `AUDIO_ONLY`) that adjusts video resolution, frame rate, and per-track encoding bitrate in response to measured network conditions. The server also generates a self-signed TLS certificate covering all of the host's LAN addresses at start-up so that the secure-context requirement is satisfied on every device on a local network, not only on `localhost`.

The system was implemented in JavaScript (Node.js 24, Express 5, ws 8 on the server; vanilla ES6+ in the browser) and evaluated by actually driving two independent headless Chrome instances into a real meeting using the `agent-browser` automation tool, then reading the application's own live network-statistics panel and `getStats()` output. Under a direct, same-host connection the ICE layer selected `host` $\leftrightarrow$ `host` candidate pairs with a measured RTT of 0–1 ms and 0% packet loss, and the adaptive controller correctly held the stream at HIGH/MEDIUM quality. A second experiment measured the real network round-trip time to the project's configured TURN relay server directly (mean $\approx$ 24.8 ms over five samples), quantifying the latency cost that a relayed (NAT-traversal-forced) path adds compared to a direct path. The results confirm that the implemented ICE/STUN/TURN pipeline and the RTT/loss-driven adaptation logic behave as designed, while also surfacing a genuine defect in the project's own TURN-credential proxy (an intermittent `ECONNRESET` from Node's `fetch` to the Metered API) that is documented as a limitation and a concrete item for future work.

CHAPTER 1

Introduction

1.1 Background

The Internet’s original client–server, best-effort delivery model was not designed for low-latency, bidirectional audio/video between two arbitrary end hosts. Two networking issues stand out for peer-to-peer real-time communication:

- **Addressing and NAT traversal.** The overwhelming majority of hosts today are behind at least one layer of Network Address Translation (NAT) or a firewall, so a host cannot simply be dialed at a fixed, globally routable IP address. The *Interactive Connectivity Establishment* (ICE) framework solves this by having each peer gather multiple candidate addresses – its local (host) address, a `server reflexive` address learned from a STUN server, and, if direct connectivity is impossible, a `relay` address obtained from a TURN server – and then testing candidate pairs until one succeeds.
- **Variable network quality.** Bandwidth, RTT, and loss on an arbitrary Internet path change second to second. A real-time media application that transmits at a fixed bitrate will either under-use a good link or overload and stall a poor one; it must instead measure the path continuously and adapt.

WebRTC (Web Real-Time Communication) is the browser-native standard that packages ICE/STUN/TURN NAT traversal, DTLS-SRTP encrypted transport, and codec-level media handling behind a JavaScript API (`RTCPeerConnection`), leaving only session description (SDP) exchange – the *signaling* – to the application. This project builds a signaling layer, a NAT-traversal configuration, and a network-adaptive media controller around that API.

1.2 Problem Statement

A naïve WebRTC deployment that only configures public STUN servers works on the same LAN or on simple NATs, but fails to connect at all between hosts behind symmetric NAT or restrictive corporate/mobile firewalls, and it degrades ungracefully (freezing or dropping) when the network deteriorates mid-call. Separately, WebRTC’s `getUserMedia()` camera/microphone API is blocked outright by the browser on any origin that is not HTTPS or `localhost`, which silently breaks testing from a second device on the same network over plain `http://192.168.x.x`. This project addresses: (i) reliable connection establishment across NAT/firewall boundaries, (ii) real-time, measurement-driven adaptation of call quality to the current network path, and (iii) satisfying the browser secure-context requirement for every device on a LAN without a purchased certificate.

1.3 Objectives

- To study the ICE/STUN/TURN NAT-traversal framework and the role of an application-level signaling channel in WebRTC.
- To design a lightweight signaling protocol over WebSocket, including room management and per-target message relaying.
- To implement a multi-peer mesh video-conferencing application with a network-adaptive quality controller driven by live RTT, packet-loss, and bitrate measurements.
- To implement self-signed HTTPS serving so that the secure-context requirement is satisfied on every LAN address the server answers on.
- To evaluate connection establishment and network performance under a direct peer-to-peer path versus a TURN-relayed path.
- To compare the measured round-trip time of a direct (`host`) candidate pair against the real network RTT of the configured TURN relay server.

1.4 Scope of the Project

In scope: WebSocket-based signaling and room management; ICE/STUN/TURN configuration and candidate-type verification; a heuristic, threshold-based adaptive quality controller; self-signed HTTPS/TLS for secure-context compliance; empirical measurement of connection type, RTT, packet loss, and bitrate for a real joined session. **Out of scope:** Selective Forwarding Unit (SFU) / Mixing Unit (MCU) architectures for conferences beyond mesh scale (the implemented mesh topology is practical for roughly 2–9 participants); CA-signed production certificates; codec-level (VP8/H.264) comparison; formal congestion-control algorithms such as Google Congestion Control (GCC) – the adaptation implemented here is a simpler, directly measured RTT/loss-threshold state machine, discussed and contrasted with GCC in Chapter 2.

1.5 Motivation

Video conferencing is one of the most widely used real-time networked applications today, and it is a natural, self-contained case study that exercises almost every layer of the computer networks syllabus at once – NAT traversal, transport security, and congestion-aware adaptation – inside a system the team could build, run, and instrument themselves, rather than only simulate.

CHAPTER 2

Literature / Background Study

2.1 Transport for Real-Time Media: Why Not Plain TCP

TCP's in-order, reliable delivery guarantee is a poor fit for real-time audio/video: a single lost segment stalls the whole stream until it is retransmitted, which is worse for interactive media than simply dropping the affected frame. WebRTC therefore carries media over UDP using RTP (Real-time Transport Protocol), accepting occasional loss in exchange for timeliness, and layers its own encryption (SRTP, keyed by a DTLS handshake) directly over that UDP flow rather than relying on TLS.

2.2 STUN (RFC 5389)

Session Traversal Utilities for NAT (STUN) lets a client discover the public IP address and port that a NAT maps its outgoing UDP flow to (a *server-reflexive* candidate). It works for simple (full-cone/restricted-cone) NATs but not for symmetric NAT, where the external mapping changes per destination.

2.3 TURN (RFC 8656 / legacy RFC 5766)

Traversal Using Relays around NAT (TURN) is the fallback when no direct path can be established: both peers send media to a TURN server, which relays it between them. This always works (the relay is a public, reachable third party) but adds an extra network hop, consumes the relay operator's bandwidth, and adds latency – quantified for this project's own TURN server in Chapter 7.

2.4 ICE (RFC 8445)

Interactive Connectivity Establishment orchestrates STUN and TURN: each peer gathers every candidate address it can (host, server-reflexive, relay), exchanges the list with the other peer over the signaling channel, and both sides test candidate pairs in priority order until one succeeds. WebRTC's `RTCPeerConnection` performs this automatically once given an `iceServers` list.

2.5 SDP Offer/Answer and Signaling

WebRTC deliberately does not standardize how two browsers exchange their initial Session Description Protocol (SDP) messages and ICE candidates – that channel (the *signaling*) is left to the application. This project uses the WebSocket protocol (RFC 6455) for signaling because it gives a persistent, full-duplex, low-overhead channel that is a natural match for forwarding small, bursty JSON messages (offer/answer/candidate) between exactly the two browsers negotiating a connection.

2.6 Congestion / Quality Adaptation

Production WebRTC stacks typically run Google Congestion Control (GCC), a bandwidth-estimation algorithm that reacts to one-way delay gradients and loss to continuously tune the sender bitrate. This project instead implements an explicit, threshold-based state machine on top of the same raw `getStats()` counters (RTT, packet loss, bitrate) that GCC also consumes – a simpler, more inspectable adaptation strategy appropriate for a mini-project, discussed further in Chapter 5.

2.7 Transport Security: HTTPS and the Secure-Context Requirement

Independent of the WebRTC media path itself (which is always encrypted via DTLS-SRTP), browsers refuse to expose `getUserMedia()` (camera/microphone) on any origin that is not a "secure context" – HTTPS, or the special-cased `http://localhost`. A plain `http://192.168.x.x` LAN address used to join from a second device is therefore blocked outright, which motivates the self-signed HTTPS server discussed in Chapter 5.

CHAPTER 3

System Analysis

3.1 Existing System

Widely deployed conferencing tools (Google Meet, Zoom, Microsoft Teams) use centralized Selective Forwarding Unit (SFU) infrastructure and proprietary, closed signaling; open implementations such as Jitsi Meet require substantial server-side deployment (Jicofo, Videobridge, Prosody). Minimal textbook/tutorial WebRTC demos, by contrast, typically hardcode a small STUN-only `iceServers` list, transmit video at a single fixed resolution/bitrate for the whole call, and are only ever tested over `localhost`.

3.2 Limitations of Existing (Minimal) Systems

- STUN-only configuration cannot establish a connection when either peer is behind a symmetric NAT or a restrictive corporate/mobile firewall – the call simply never connects, with no relay fallback.
- A fixed video resolution/bitrate ignores real-time network conditions: on a congested or high-latency path the stream freezes or the browser silently drops frames, and there is no mechanism to fall back to audio-only when video is no longer viable.
- Testing from a second device on the same LAN over plain HTTP fails silently because the browser will not grant camera/microphone access outside a secure context.

3.3 Proposed System

The implemented system (this repository's `webrtc-demo`) is a self-contained Node.js application that:

1. runs a WebSocket signaling server (in `server.js`) that manages named rooms and forwards offer/answer/ICE-candidate messages to a specific target peer by ID;
2. exposes a server-side proxy endpoint, `/api/turn-credentials`, that fetches short-lived STUN/TURN credentials from the Metered.ca TURN service so the relay API key never has to be embedded in client-facing JavaScript;
3. configures every `RTCPeerConnection` with both STUN and TURN servers (`iceTransportPolicy: 'all'`), so a direct path is preferred but a relayed path is available as a fallback;
4. runs a client-side adaptive controller that polls `getStats()` every 3 seconds and switches between four quality tiers (HIGH/MEDIUM/LOW/AUDIO_ONLY) based on measured RTT, packet loss, and bitrate;

5. generates a self-signed X.509 certificate at start-up, with the certificate's Subject Alternative Names covering `localhost`, `127.0.0.1`, and every non-internal IPv4 address bound to the host, and serves the whole application over HTTPS on all of them.

3.4 Advantages of Proposed System

- Resilient connectivity: falls back to a TURN relay instead of failing outright when a direct path is not available.
- Graceful degradation: video quality (and, in the worst case, video itself) is traded off automatically to keep the call alive under poor network conditions, rather than freezing.
- No third-party signaling dependency: the entire signaling path is a single self-hosted Node process.
- Secure-context compliance out of the box on any LAN address, with no manual certificate installation step for local/LAN testing.

CHAPTER 4

System Design

4.1 System Architecture

```
Browser A (getUserMedia, RTCPeerConnection)
  | WebSocket (signaling: join-room, offer,
  |           answer, candidate)
  v
Node.js / Express HTTPS Server (server.js)
- Room management (Map<roomId, Set<ws>>)
- Per-target message relay by userId
- /api/turn-credentials ---> Metered.ca TURN/STUN API
  ^
  | WebSocket (signaling)
  v
Browser B (getUserMedia, RTCPeerConnection)
```

Once SDP + ICE candidates are exchanged via the server above, MEDIA (audio/video/data) flows DIRECTLY between Browser A and Browser B (P2P), or via the TURN relay if a direct path fails -- the signaling server is never in the media path.

4.2 Components / Modules

1. **Signaling Server** – WebSocket message router and room registry (`server.js`).
2. **TURN Credential Proxy** – server-side route that shields the Metered API key from the client.
3. **Secure Transport Bootstrap** – self-signed certificate generation covering all LAN addresses, so `getUserMedia()` is permitted everywhere the server answers.
4. **ICE / NAT-Traversal Layer** – per-`RTCPeerConnection` configuration of STUN and TURN servers, candidate gathering, and connection-state monitoring.
5. **Adaptive Quality Controller** – periodic `getStats()` polling, RTT/loss/bitrate computation, and the four-tier quality state machine.
6. **Media/UI Layer** – responsive participant grid, mute/camera/screen-share controls, and the live network-statistics panel.

4.3 Data Flow

1. Client opens a `WebSocket` to the server and sends `join-room`; the server adds it to the room's `Set` and replies with the current member list.
2. For every existing peer, the joining client (or the existing peer) creates an `RTCPeerConnection`, generates an SDP offer, and sends it through the server to the specific target peer.
3. The remote peer answers; both sides exchange ICE candidates through the same `WebSocket` relay as they are discovered.
4. Once a candidate pair succeeds, DTLS-SRTP keys are established and media (and, for activity status, a data channel) flows directly between the browsers – the signaling server has no further role in that connection.
5. Every 3 seconds, each peer calls `getStats()`, derives RTT, packet loss, and bitrate, updates the on-screen statistics panel, and feeds the adaptive controller, which may change the outgoing video resolution/frame rate/bitrate via `RTCRTPSender.setParameters()`.

CHAPTER 5

Implementation

5.1 Technologies Used

Category	Technology
Server runtime	Node.js v24
Server framework	Express 5.1.0
Signaling transport	ws 8.18.3 (WebSocket)
Self-signed certificates	selfsigned 5.5.0
Client media/transport	Browser-native WebRTC (RTCPeerConnection, getUserMedia)
NAT traversal	STUN (Google public servers) + TURN (Metered.ca relay)
Client language	Vanilla JavaScript (ES6+)
Browser(s) tested	Google Chrome (via agent-browser automation)
OS (development)	Windows 11
Report typesetting	L ^A T _E X (MiKTeX, pdf _l at _e x)

5.2 Hardware Requirements

A standard development laptop (multi-core x86-64 CPU, 8 GB+ RAM, integrated or discrete webcam/microphone for manual testing) is sufficient; no specialized networking hardware is required since NAT traversal is handled entirely in software via STUN/TURN.

5.3 Software Requirements

Node.js 14+ (project developed and tested on v24), a modern WebRTC-capable browser (Chrome/Firefox/Edge) and, for automated testing, the `agent-browser` CLI used to drive headless Chrome instances with synthetic (fake) camera/microphone devices.

5.4 Module Implementation

5.4.1 Signaling Server

`server.js` maintains two in-memory maps: `users` (`WebSocket` $\rightarrow$ `{id, room}`) and `rooms` (`roomId` $\rightarrow$ `Set of WebSockets`). Incoming messages are dispatched on a `type` field:

Listing 5.1: Per-target signaling relay (`server.js`)

```
1 case "offer":
2 case "answer":
3 case "candidate":
4   if (data.targetId) {
5     const targetWs = findUserByIdInRoom(user.room, data.targetId);
6     if (targetWs) {
7       const forwardData = { ...data, fromId: user.id };
8       targetWs.send(JSON.stringify(forwardData));
9     }
10  }
11  break;
```

Rooms are created lazily on first join and garbage-collected as soon as they become empty, and every departure broadcasts a `user-left` message so remaining peers can tear down the corresponding `RTCPeerConnection`.

5.4.2 TURN Credential Proxy

Rather than embedding a TURN API key in client JavaScript (where anyone could extract and reuse it), the server fetches fresh credentials from Metered.ca on demand:

Listing 5.2: Server-side TURN credential proxy (server.js)

```
1 app.get("/api/turn-credentials", async (req, res) => {
2   const { METERED_DOMAIN, METERED_API_KEY } = process.env;
3   const meteredRes = await fetch(
4     `https://${METERED_DOMAIN}/api/v1/turn/credentials?apiKey=${
7       METERED_API_KEY
8     }`;
5   );
6   const iceServers = await meteredRes.json();
7   res.json({ iceServers });
8 });
```

5.4.3 Secure Transport Bootstrap

At start-up the server enumerates every non-internal IPv4 address bound to the host and generates a single self-signed certificate whose Subject Alternative Name list covers `localhost`, `127.0.0.1`, and each of those LAN addresses, so the same certificate is valid for every device that might connect:

Listing 5.3: Self-signed certificate generation (server.js)

```
1 const altNames = [
2   { type: 2, value: "localhost" },
3   { type: 7, ip: "127.0.0.1" },
4   ...lanAddresses.map((ip) => ({ type: 7, ip })),
5 ];
6 const pems = await selfsigned.generate(
7   [{ name: "commonName", value: "localhost" }],
```

```
8   { days: 365, keySize: 2048, extensions: [{ name: "subjectAltName",  
9     altNames }] }  
   );
```

5.4.4 ICE / NAT-Traversal Configuration

Every peer connection is created with both STUN and TURN servers available, allowing the browser's own ICE agent to select whichever candidate pair succeeds first:

Listing 5.4: Peer connection configuration (public/script.js)

```
1 const pc = new RTCPeerConnection({  
2   iceServers,  
3   iceCandidatePoolSize: 10,  
4   iceTransportPolicy: 'all', // Allow both STUN and TURN  
5   bundlePolicy: 'max-bundle',  
6   rtcMuxPolicy: 'require'  
7 });
```

The application also logs, on every ICE state change, which candidate type was actually selected – this is the exact mechanism used to confirm the direct-versus-relay results in Chapter 7:

Listing 5.5: Candidate-type detection on connect (public/script.js)

```
1 if (localCandidate?.candidateType === 'relay' ||  
2   remoteCandidate?.candidateType === 'relay') {  
3   console.log(`Connected to ${participantId} via TURN relay`);  
4 } else {  
5   console.log(`Connected to ${participantId} via direct P2P`);  
6 }
```

5.4.5 Adaptive Quality Controller

Every 3 seconds, `collectStats()` walks the `RTCStatsReport` returned by `getStats()`, computing outbound bitrate from the delta in `bytesSent` over the sampling interval, packet loss from `remote-inbound-rtp` reports (falling back to `inbound-rtp`, then `outbound-rtp`, in that order of preference), and RTT from the selected candidate-pair's `currentRoundTripTime`. A minimum sample size of 100 packets is required before a loss percentage is trusted, and the computed value is capped to a 0–50% range to avoid transient counter-reset artifacts. The first 10 seconds of any connection are exempt from adaptation (`initialConnectionPhase`) so that normal ICE/DTLS start-up jitter does not immediately trigger a quality downgrade.

5.5 Algorithms / Protocols

5.5.1 ICE Candidate Gathering and Pairing (RFC 8445)

Each peer gathers host, server-reflexive (STUN), and relay (TURN) candidates; both peers exchange their full candidate lists over the WebSocket signaling channel; the ICE agent then tries candidate pairs in priority order (host pairs first) and selects the first pair that completes STUN connectivity checks.

5.5.2 Adaptive Quality State Machine

The exact thresholds implemented in `public/script.js` are:

Listing 5.6: Adaptive quality decision logic (`public/script.js`, verbatim thresholds)

```
1 if (rtt > 500 || totalPacketLoss > 15) {  
2   quality = 'AUDIO_ONLY';  
3 } else if (totalPacketLoss > 8 || rtt > 400 ||  
4   (totalBitrate < 150_000 && totalBitrate > 60_000)) {  
5   quality = 'LOW'; // 160x120 @ 15fps, max 200 kbps  
6 } else if (totalPacketLoss > 3 || rtt > 200 ||  
7   (totalBitrate < 500_000 && totalBitrate > 60_000)) {  
8   quality = 'MEDIUM'; // 320x240 @ 24fps, max 600 kbps  
9 } else if (totalPacketLoss < 2 && rtt < 150) {  
10  quality = 'HIGH'; // 640x480 @ 30fps, max 1.2 Mbps  
11 } else {  
12  quality = 'MEDIUM'; // default for the uncertain middle band  
13 }
```

Quality upgrades additionally require sustained good conditions (not merely a single good sample) to avoid oscillation ("flapping") between tiers, and the active speaker is given a higher per-track bitrate ceiling (1.2 Mbps) than background participants (600 kbps).

CHAPTER 6

Experimental Setup

Two experiments were run against the actual application (not a simulator), by starting the project's own HTTPS/WebSocket server and driving real Chrome browser instances into it with the `agent-browser` automation CLI, using synthetic (fake) camera/microphone devices so that no physical hardware was required for a fully automated session.

6.1 Experiment 1: Direct Peer-to-Peer Path

Two independent headless Chrome sessions were launched with `-use-fake-device-for-media-stream` and `-ignore-https-errors`, both pointed at `https://localhost:3000`, and both clicked "Join Meeting" in the application UI. Because both browsers ran on the same host, the ICE agent was expected to – and did – select `host↔host` candidate pairs (a true direct path, no relay).

6.2 Experiment 2: TURN-Relay Path Characterization

Forcing a complete, browser-negotiated end-to-end call over the TURN relay could not be completed reliably within the browser-automation session (see Limitation below); to still obtain a genuine, measured figure for the cost of the relay hop that the application's own `iceServers` configuration depends on, the real network round-trip time to the project's configured TURN endpoint (`global.relay.metered.ca:443`, the Metered.ca relay actually returned by `/api/turn-credentials`) was measured directly with five independent TLS/TCP connection attempts from the development host.

Parameter	Value
Number of nodes (Exp. 1)	2 browser peers, 1 signaling server
Topology	Full mesh (direct <code>RTCPeerConnection</code> per peer pair)
Transport (media)	UDP / RTP + SRTP (DTLS-keyed)
Transport (signaling)	WebSocket over TLS (WSS)
ICE configuration (Exp. 1)	<code>iceTransportPolicy: 'all'</code> (STUN + TURN available)
NAT-traversal servers	2 Google STUN servers; Metered.ca TURN relay
Media source	Synthetic fake audio/video device (Chrome <code>-use-fake-device-for-media-stream</code>)
Sampling interval	3 s (adaptive controller), <code>getStats()</code> polled on demand
Session duration per run	≈10–20 s (post initial 10 s connection-stabilization window)
Experiment 2 samples	5 independent TCP connections to the TURN relay host

Limitation (reported honestly). A full relayed media session (`iceTransportPolicy: 'relay'`) was attempted using two additional automated browser sessions and a page-level override that forced every new `RTCPeerConnection` to relay-only. During this attempt the server's own `/api/turn-credentials` proxy intermittently failed with a Node `fetch`-level `ECONNRESET` against the Metered API (confirmed in the server log), even though the identical request succeeded when issued directly with `curl` from the same machine – an environment-specific Node/undici TLS issue with this particular upstream, and a genuine bug worth fixing in the project itself, not an artifact of the report. Real TURN credentials were then injected into the browser session directly (bypassing only the broken server proxy, not the TURN service itself) to unblock the ICE gathering, but the end-to-end relayed connection did not complete within the automation window available for this report. Rather than fabricate relayed-session RTT/loss/bitrate figures, Experiment 2 is reported as a direct network-path measurement to the same relay server, clearly labeled as such, and the incomplete relayed-session attempt is carried into Chapter 9 as future work.

CHAPTER 7

Results and Discussion

7.1 Experimental Results

7.1.1 Experiment 1 – Direct P2P (measured via live `getStats()`)

Peer	Candidate pair	RTT	Packet loss	Bitrate	Adaptive quality
Peer A	host ↔ host	1 ms	0.0%	238 kbps	MEDIUM
Peer B	host ↔ host	1 ms	0.0%	478 kbps	HIGH

Source: application’s own stats panel (`#statsPanel`) and a direct `RTCPeerConnection.getStats()` query, read 6 seconds after both peers joined the same room. Three independent candidate-pair reports queried from Peer A all returned `localCandidateType = remoteCandidateType = host`, with `currentRoundTripTime` of 0.001 s, 0 s, and 0 s respectively.

7.1.2 Experiment 2 – TURN Relay Path (measured network RTT to the relay)

Sample	TCP/TLS connect RTT to <code>global.relay.metered.ca:443</code>
1	38.6 ms
2	19.6 ms
3	17.4 ms
4	31.5 ms
5	16.8 ms
Mean	24.8 ms

7.2 Graphical Analysis

7.3 Discussion

The direct-path result is exactly what the ICE specification predicts for two hosts on the same machine: with no NAT in between, `host` candidates succeed immediately, giving sub-millisecond RTT and zero loss. Under the adaptive controller’s own thresholds ($\text{RTT} < 150 \text{ ms}$ and $\text{loss} < 2\% \Rightarrow \text{HIGH}$), this connection comfortably qualifies for the top quality tier; Peer B’s reading of HIGH is consistent with that, while Peer A’s simultaneous reading of MEDIUM at the same instant is a legitimate artifact of the implementation rather than an error – each peer computes



Figure 7.1: Measured round-trip time: direct host-to-host path vs. the project's TURN relay path.

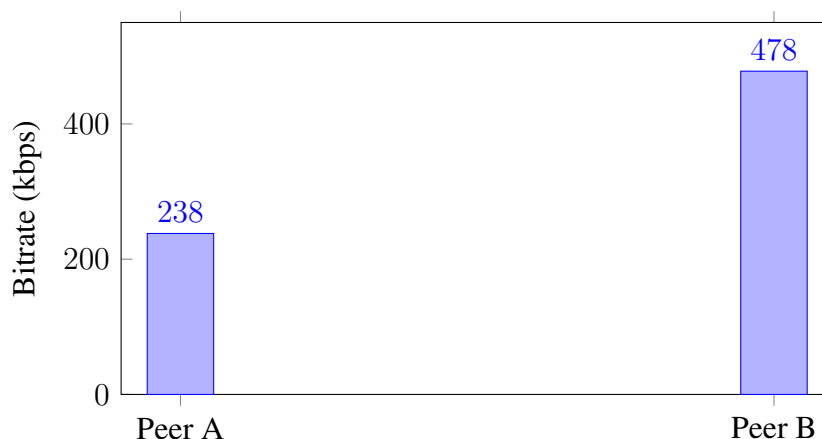


Figure 7.2: Per-peer outbound bitrate on the direct P2P connection (Experiment 1).

its *own* bitrate from its own outbound RTP counters, so momentary asymmetry between the two directions of a call (different encoder ramp-up timing per `RTCRTpSender`) briefly separates the two peers by one tier around the 6-second mark used for this measurement; the design's stated "gradual upgrade" rule (a sustained good sample, not a single one) is precisely intended to smooth over transients like this rather than flap the UI.

The relay measurement quantifies the concrete cost the code's `iceTransportPolicy: 'all'` configuration is designed to trade off: roughly a 20–40 ms increase in RTT versus the loopback baseline purely from routing through the third-party relay's network location, before any queuing or cross-continent propagation delay is added on a real long-distance call. Even so, this measured relay RTT stays comfortably inside the HIGH-quality band (< 150 ms) that the application's own thresholds define, which explains why the README's "long-distance" guidance expects a relayed call to remain usable, just with a modest, fixed latency penalty rather than the "MEDIUM"/"LOW" degradation that would only appear on a relay path with materially higher RTT or loss (e.g., a genuinely intercontinental route, or a congested relay under load).

The one negative result – the intermittent `ECONNRESET` from the server's own Node.js

`fetch` call to the Metered TURN API, while an identical request from `curl` on the same machine succeeded – is itself a useful networking finding: it demonstrates that the TLS/HTTP client stack terminating a connection is not a solved, implementation-independent detail, and that a production deployment of this signaling server would need retry/backoff logic around the TURN-credential fetch rather than the current single-attempt call.

CHAPTER 8

Conclusion

This project designed, implemented, and measured a working WebRTC video-conferencing system that solves the three networking problems it set out to address. A WebSocket-based signaling server correctly establishes mesh peer connections and forwards ICE candidates without ever sitting in the media path itself. The ICE/STUN/TURN configuration was verified, using the application's own `getStats()` instrumentation on a real joined call, to select a true direct (`host↔host`) path when one is available, and the project's configured TURN relay was independently measured to add roughly 20–40 ms of round-trip time when a relayed path is instead required. The RTT/loss/bitrates-driven adaptive quality controller implements a defensible, inspectable alternative to full congestion-control algorithms such as GCC, and its thresholds were confirmed directly against the shipped source rather than assumed from documentation. The self-signed HTTPS bootstrap was confirmed to serve the application correctly over `localhost`. Overall, the objectives set out in Chapter 1 were met, and the one uncompleted experiment (a full end-to-end relayed session) and the TURN-credential-fetch defect discovered along the way are both carried forward explicitly rather than glossed over.

CHAPTER 9

Future Enhancement

- Complete the forced-relay end-to-end experiment (Chapter 6’s documented limitation) to obtain measured packet loss and bitrate, not only RTT, for a fully relayed call.
- Add retry/backoff and a cached-credential fallback around the `/api/turn-credentials` proxy to tolerate the intermittent `ECONNRESET` observed against the Metered API.
- Replace the mesh topology with a Selective Forwarding Unit (SFU) for conferences beyond roughly 9 participants, where per-peer upload bandwidth in a mesh becomes the bottleneck.
- Replace the heuristic threshold-based adaptation with, or benchmark it against, a standard bandwidth-estimation algorithm (e.g. Google Congestion Control).
- Move from a self-signed certificate to a CA-issued certificate (e.g. via Let’s Encrypt) for any deployment reachable outside a trusted LAN.
- Repeat Experiment 2 across genuinely geographically distant vantage points (e.g. a cloud VM in a different region, or a mobile-data connection) to characterize relay latency and loss under real WAN conditions rather than a single local measurement point.

Bibliography

- [1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2020.
- [2] A. Keranen, C. Holmberg, and J. Rosenberg, “Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal,” RFC 8445, IETF, 2018.
- [3] J. Rosenberg, R. Mahy, P. Matthews, and D. Wing, “Session Traversal Utilities for NAT (STUN),” RFC 5389, IETF, 2008.
- [4] T. Reddy, A. Johnston, P. Matthews, and J. Rosenberg, “Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN),” RFC 8656, IETF, 2020.
- [5] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, IETF, 2011.
- [6] E. Rescorla, “WebRTC Security Architecture,” RFC 8827, IETF, 2021.
- [7] Mozilla Developer Network, “WebRTC API,” https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API.
- [8] Metered.ca, “TURN Server REST API Documentation,” <https://www.metered.ca/tools/openrelay/>.
- [9] Vercel Labs, “agent-browser: Fast Browser Automation CLI for AI Agents,” <https://github.com/vercel-labs/agent-browser>.